Karthik Ayyer

Revenue Lifecycle Orchestration & Data Governance Framework

· karthik.kannan0590@gmail.com · linkedin.com/in/karthik-ayyer

This document outlines an example framework I've used to think through scalable GTM systems architecture across enterprise marketing platforms. The objective is to standardize lifecycle orchestration, identity resolution, routing, enrichment, integration reliability, and operational observability while supporting scalable global demand generation operations.

Section A: Framework Overview

This framework represents how I typically think about scalable GTM systems architecture across integrated marketing platforms. The objective is to create reliable lifecycle orchestration across ingestion, normalization, identity resolution, routing, activation, and reporting layers while maintaining operational governance and observability.

The framework is designed to address common operational challenges across enterprise GTM ecosystems, including:

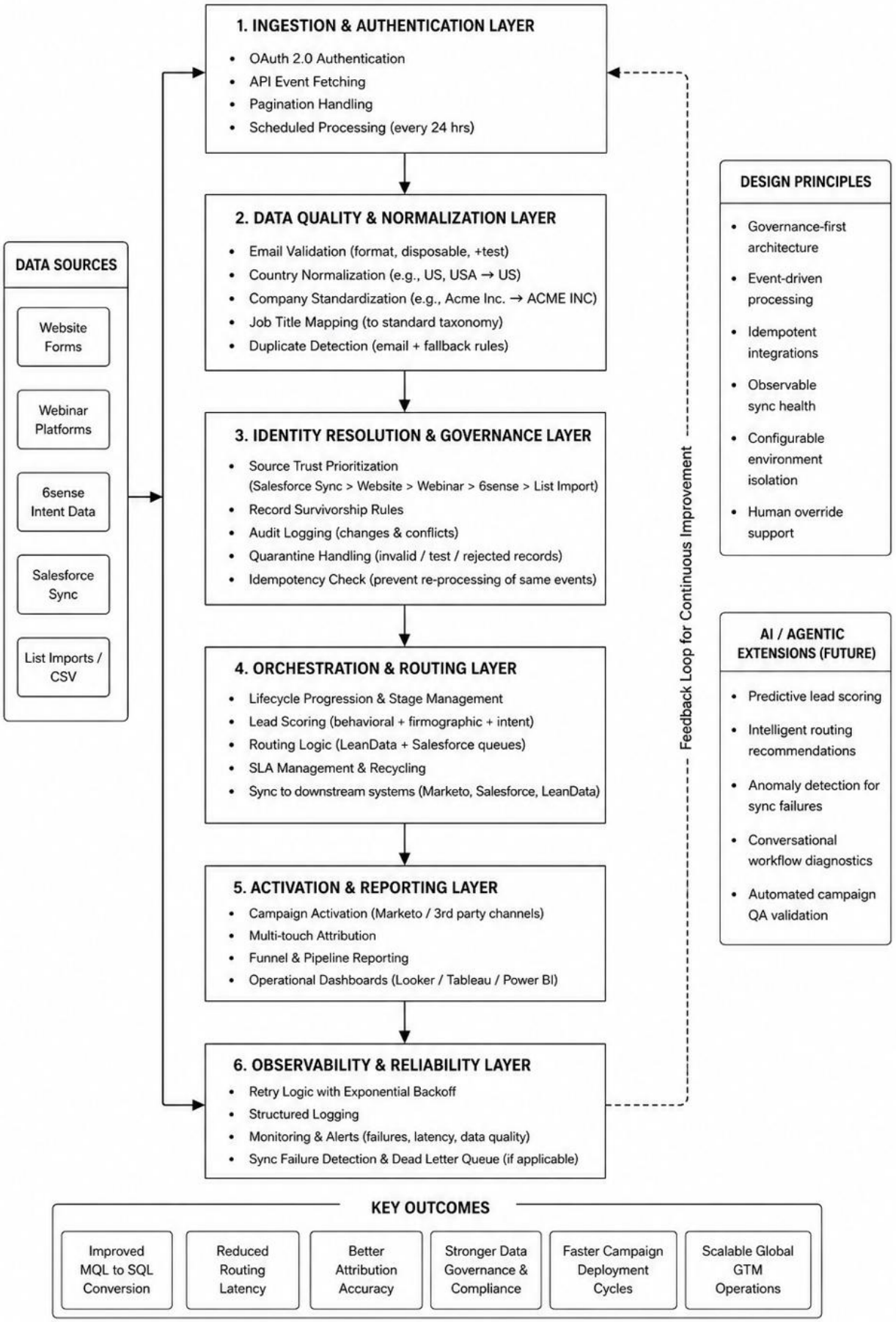
- Duplicate and fragmented lead/account records across systems
- Inconsistent lifecycle progression and routing behaviour
- Limited attribution visibility across channels and touchpoints
- Data normalization issues impacting segmentation and reporting
- Operational inefficiencies caused by sync failures and manual intervention
- Scalability challenges across global and regional marketing programs
- Lack of observability across integrated platform workflows

A1. Architectural Scope

The framework is structured across the following operational layers:

- Ingestion & Authentication
- Data Quality & Normalization
- Identity Resolution & Governance
- Lifecycle Orchestration & Routing
- Activation & Reporting
- Observability & Reliability

Each layer is designed to operate independently while contributing to an end-to-end lifecycle orchestration model.


1. INGESTION & AUTHENTICATION LAYER
OAuth 2.0 Authentication
API Event Fetching
Pagination Handling
Scheduled Processing (every 24 hrs)
2. DATA QUALITY & NORMALIZATION LAYER
Email Validation (format, disposable, +test)
Country Normalization (e.g., US, USA → US)
Company Standardization (e.g., Acme Inc. → ACME INC)
Job Title Mapping (to standard taxonomy)
Duplicate Detection (email + fallback rules)
3. IDENTITY RESOLUTION & GOVERNANCE LAYER
Source Trust Prioritization (Salesforce Sync > Website > Webinar > 6sense > List Import)
Record Survivorship Rules
Audit Logging (changes & conflicts)
Quarantine Handling (invalid / test / rejected records)
Idempotency Check (prevent re-processing of same events)
4. ORCHESTRATION & ROUTING LAYER
Lifecycle Progression & Stage Management
Lead Scoring (behavioral + firmographic + intent)
Routing Logic (LeanData + Salesforce queues)
SLA Management & Recycling
Sync to downstream systems (Marketo, Salesforce, LeanData)
5. ACTIVATION & REPORTING LAYER
Campaign Activation (Marketo / 3rd party channels)
Multi-touch Attribution
Funnel & Pipeline Reporting
Operational Dashboards (Looker / Tableau / Power BI)
6. OBSERVABILITY & RELIABILITY LAYER
Retry Logic with Exponential Backoff
Structured Logging
Monitoring & Alerts (failures, latency, data quality)
Sync Failure Detection & Dead Letter Queue (if applicable)
DATA SOURCES
Website Forms
Webinar Platforms
6sense Intent Data
Salesforce Sync
List Imports / CSV
Feedback Loop for Continuous Improvement
DESIGN PRINCIPLES
Governance-first architecture
Event-driven processing
Idempotent integrations
Observable sync health
Configurable environment isolation
Human override support
AI / AGENTIC EXTENSIONS (FUTURE)
Predictive lead scoring
Intelligent routing recommendations
Anomaly detection for sync failures
Conversational workflow diagnostics
Automated campaign QA validation
KEY OUTCOMES
Improved MQL to SQL Conversion
Reduced Routing Latency
Better Attribution Accuracy
Stronger Data Governance & Compliance
Faster Campaign Deployment Cycles
Scalable Global GTM Operations

Section B: Data Quality, Identity Resolution & Routing Strategy

B1. Data Hygiene Strategy

Deduplication Rules

The primary deduplication key is a case-insensitive, trimmed email address. Email is the most reliable unique identifier for a person across sources. Secondary signals (full name + company domain) serve as a fuzzy fallback for records with missing or invalid emails.

Surviving record selection follows this priority order:

- Exclude or deprioritize records with invalid or test emails (e.g., invalid-email, +test aliases) - these are quarantined first.
- Among valid records, prefer the highest-trust source: Website and Salesforce Sync are most reliable; intent platforms like 6sense come next; list imports are lowest trust. (e.g., Salesforce Sync > Webinar > 6sense > Website > List Import).
- Within the same source tier, keep the most recently created record.
- Conflicting values from non-surviving records are written to an audit table so no data is permanently lost.

Normalization Approach

- **Country:** Maintain a lookup table covering all variants (US, USA, United States) and normalize to ISO codes (e.g., US, GB, DE). Enforced via Marketo smart campaign on record create/update and RingLead normalization rules at point of entry.
- **Company Name:** Strip legal suffixes (Inc, LLC, Ltd), remove punctuation, and UPPER-CASE before comparing. Fuzzy match (≥85% token sort ratio) handles variants like "Acme", "Acme Inc", and "ACME INC." The canonical form is stored; the original is preserved for audit.
- **Job Title:** Map raw titles to a standard seniority × function taxonomy. A keyword mapping table resolves "VP, Information Technology", "VP IT", and "Vice President of IT" all to VP | IT. Unmapped titles are flagged for manual review.

Validation Rules & Handling

- **Invalid email format:** Record is quarantined in a holding table with a rejection_reason. Not routed to Salesforce. Source owner is notified.
- **Test or internal emails:** +test aliases and internal domains (e.g. @mailinator.com) are flagged as TEST and excluded from routing and scoring.
- **Missing required fields:** Records with null emails are not created in CRM. Missing company/country trigger a ZoomInfo enrichment request before the record is released downstream.

In RingLead, these rules are enforced as field validation at the point of merge/dedupe. In Marketo, a data quality smart campaign fires on 'Lead is Created' and 'Data Value Changes' to catch bad data from any entry source.

B2. SQL Query – Deduplication Logic

The query below identifies duplicates by normalized email, selects the surviving record using source trust and recency, applies normalization inline, and flags invalid emails all in a single pass.

```
-- Part 1 CTE Setup

WITH ranked_leads AS (
  SELECT
    lead_id,
    LOWER(TRIM(email))                          AS email_norm,
    first_name,
    last_name,
    -- Normalize country to ISO 2-char code
    CASE
      WHEN UPPER(TRIM(country)) IN ('US','USA','UNITED STATES') THEN 'US'
      ELSE UPPER(TRIM(country))
    END                                          AS country_norm,
    -- Normalize company name
    REGEXP_REPLACE(UPPER(TRIM(company)), r'[.,\s]+(INC|LLC|LTD)\.?$', '') AS company_norm,
    title,
    created_at,
    source,
    -- Score sources: Website & Salesforce > intent tools > list imports
    CASE source
      WHEN 'Salesforce Sync' THEN 1
      WHEN '6sense'          THEN 2
      WHEN 'Webinar'         THEN 3
      WHEN 'Website'         THEN 4
      ELSE 5
    END                                          AS source_rank,
    -- Flag invalid emails upfront
    CASE
      WHEN email NOT LIKE '%@%.%'  THEN 'INVALID'
      WHEN email LIKE '%+test%'    THEN 'TEST'
      ELSE 'VALID'
    END                                          AS email_status,
    ROW_NUMBER() OVER (
      PARTITION BY LOWER(TRIM(email))
      ORDER BY
          -- Deprioritize invalid/test emails
        CASE WHEN email NOT LIKE '%@%.%' OR email LIKE '%+test%' THEN 1 ELSE 0 END ASC,
        source_rank ASC,                      -- prefer high-trust sources
        created_at DESC                       -- tiebreak: most recent
     )                                            AS rn
  FROM leads
  WHERE email IS NOT NULL
  AND email != ''
),
-- Part 2 Surviving CTE (Pick the Winner)
surviving AS (
  SELECT * FROM ranked_leads WHERE rn = 1
)
-- Part 3 Final Select
SELECT
  s.lead_id,
  s.email_norm                                   AS email,
  INITCAP(s.first_name)                          AS first_name,
  INITCAP(s.last_name)                           AS last_name,
  s.country_norm                                 AS country,
  s.company_norm                                 AS company,
  s.title, s.created_at, s.source, s.email_status,
  -- Count of dupes that were collapsed
  COUNT(*) OVER (PARTITION BY s.email_norm) - 1  AS duplicate_count
FROM surviving s
ORDER BY s.created_at DESC;
```

CTE / step	What it does
ranked_leads	Normalizes email (LOWER/TRIM), country (CASE mapping), company (strip legal suffixes). Assigns source_rank (1=Website → 5=List Import), email_status (VALID/TEST/INVALID), and ROW_NUMBER() partitioned by normalized email.
surviving	Keeps only rn = 1 — the single best record per email group based on email validity, source trust, and recency.
Final SELECT	Returns cleaned fields with INITCAP name casing and a duplicate_count column showing how many records were collapsed into each survivor.

Section C: Scripting & API Integration

C1. API Integration Design – End-to-End Flow

I would think of this as a lightweight service that runs on a schedule and processes intent data reliably without creating duplicates or gaps.

The script runs every 24 hours. Flow: Load config → get_token → fetch_events (paginated) → transform → post_with_retry → idempotency check in main loop.

Step	What happens & why
1. Load config	All credentials and settings (API URLs, client ID, client secret, page size) are read from environment variables using os.getenv(). Nothing sensitive is hardcoded. The same script runs in sandbox or production by simply swapping the env file.
2. Authenticate (get_token)	The script posts the client ID and secret to the token endpoint and gets back an OAuth 2.0 access token. This token is passed as a Bearer header on every subsequent API call. If authentication fails, the job fails immediately. No point fetching data without a valid token.
3. Fetch events with pagination (fetch_events)	The 6sense API returns results in pages. The script requests page 1, then page 2, and so on, collecting all results into a single list. It stops when a page returns fewer results than PAGE_SIZE, which signals there are no more pages. This is safe pagination; it never misses data and never makes an extra unnecessary call.
4. Transform (transform)	Each raw event is mapped to the target schema, extracting event_id, score, and timestamp. This normalizes the data into a clean, consistent format before it is sent downstream. Invalid or missing fields are handled gracefully with .get() so one bad event doesn't crash the loop.
5. Post with retry (post_with_retry)	Each transformed payload is posted to the downstream system (e.g. Marketo). If the request fails, the script retries up to 3 times using exponential backoff, waiting 1s, then 2s, then 4s between attempts. This handles transient network or server errors without manual intervention.
6. Idempotency (main loop)	Before processing each event, the script checks if its ID is already in processed_ids. If yes, it skips it. If the post succeeds, the ID is added to the set. This ensures the same event is never sent twice, even if the script reruns due to a failure or scheduling overlap.

C2. Example Code (Python-style pseudocode)

Python-style pseudocode. Logic is kept simple and readable.

```
import os
import time
import requests

API_URL       = os.getenv("API_URL")
TOKEN_URL     = os.getenv("TOKEN_URL")
CLIENT_ID     = os.getenv("CLIENT_ID")
CLIENT_SECRET = os.getenv("CLIENT_SECRET")
PAGE_SIZE     = int(os.getenv("PAGE_SIZE", "200"))

processed_ids = set()   # tracks already-processed event IDs (idempotency)

def get_token():
    response = requests.post(TOKEN_URL, data={
        "client_id":     CLIENT_ID,
        "client_secret": CLIENT_SECRET,
        "grant_type":    "client_credentials"
    })
    return response.json()["access_token"]


def fetch_events(token):
    headers    = {"Authorization": f"Bearer {token}"}
    all_events = []
    page       = 1

    while True:
        response = requests.get(f"{API_URL}/events",
                        headers=headers,
                        params={"page": page}).json()
        all_events.extend(response)
        if len(response) < PAGE_SIZE:   # fewer results = no more pages
            break
        page += 1

    return all_events


def transform(event):
    return {
        "event_id":  event["id"],
        "score":     event.get("score"),
        "timestamp": event.get("timestamp")
    }


def post_with_retry(payload):
    for i in range(3):   # retry up to 3 times with exponential backoff
        try:
            r = requests.post("DOWNSTREAM_URL", json=payload)
            if r.status_code < 300:
                return True
        except:
            time.sleep(2 ** i)   # waits 1s, 2s, 4s between retries
    return False
```

```
def main():
    token  = get_token()
    events = fetch_events(token)

    for event in events:
        if event["id"] in processed_ids:
            continue   # skip — already processed

        payload = transform(event)
        success = post_with_retry(payload)

        if success:
            processed_ids.add(event["id"])

if __name__ == "__main__":
    main()
```

Notes

- Retry uses exponential backoff (1s → 2s → 4s between attempts).
- Idempotency: processed_ids tracks event IDs already sent. This prevents duplicates if the script reruns.
- All config (URLs, credentials, environment) comes from environment variables - no hardcoding.